

Chapter 3

Solving Problems by Searching and Constraint Satisfaction Problem [SEng9141]

Learning Objectives

By the end of this chapter, students will be able to:

- Formulate problems as search tasks.
- Compare search strategies.
- Apply constraint satisfaction techniques.
- Model games as search problems.

Basic outline

1. Problem Solving by Searching
2. Problem Solving Agents
3. Problem Formulation
4. Search Strategies
5. Avoiding Repeated States
6. Constraint Satisfaction Search
7. Games as Search Problems

3.1 Introduction to Problem-Solving Agents

What is a Problem-Solving Agent?

- ❖ A problem-solving agent is a type of goal-based artificial intelligence that determines **a sequence of actions** to achieve a specific objective, often using **search algorithms** to explore potential solutions.
- ❖ It operates by formulating goals, defining problems (initial states, actions, tests), and planning steps to reach a desired state, commonly used in navigation and robotics.

Components of Problem-Solving Agent

- ❖ Goal Formulation: What states are desirable?
- ❖ Problem Formulation: What actions and states to consider?
- ❖ Search: Finding action sequences that lead to goal states
- ❖ Execution: Carrying out the solution

What is an Agent? (recall)

- An agent is a system that perceives its environment through sensors, processes information, and takes autonomous actions to achieve specific goals.
- Agents can range from simple reflex systems that react instantly to complex, proactive systems that plan for the future.
- **Perception**: Gathers information about the environment.
- **Action**: Executes actions to interact with the environment.
- **Autonomy**: Operates without direct human control.

Cont....

- A problem-solving agent is a goal-based agent that maps out a **sequence of actions** to reach a desired state rather than reacting solely to current stimuli.
- It is specialized for situations requiring planning and foresight, such as puzzle-solving, navigating paths, or strategic decision-making.

3.2 Characteristics and Components

- **Goal Formulation:** Defines the desired target state.
- **Problem Formulation:** Identifies the initial state, actions, transition model, and goal test.
- **Search:** Explores state space using algorithms (e.g., Breadth-First, Depth-First) to find the best path.
- **Execution Phase:** Carries out the determined sequence of actions.
- **Performance Measure:** Often optimizes for the lowest path cost to the goal.

Examples

- **GPS Navigation:** Finding the shortest path between locations.
- Game Playing AI: Planning moves in Chess or Go.
- Robotics: Path planning and navigation.

Problem-Solving Agent vs. Simple Agent:

- While a basic reflex agent only reacts to the current situation, a problem-solving agent considers the future consequences of a sequence of actions.

Example: The 8-Puzzle

- Initial State: Goal State:

1 2 3

1 2 3

4 - 5

4 5 6

7 8 6

7 8

Formal Definition:

- States: Configurations of the puzzle
- Initial State: Starting configuration
- Actions: Move blank up, down, left, right
- Transition Model: $\text{Result}(\text{state}, \text{action}) \rightarrow \text{new state}$
- Goal Test: Check if state matches goal configuration
- Path Cost: Each move costs 1

Cont....

- Problem solving is a fundamental concept in artificial intelligence (AI) where systems are designed to identify challenges, make decisions and find efficient solutions.
- AI uses agents which are systems that perceive their environment and take actions to achieve specific goals.
- They go beyond simple reflex agents which react to stimuli based on pre-defined rules.
- Instead, problem-solving agents actively analyze situations, evaluate different options and choose the best action to reach their goal.

These agents work by:

- **Perceiving the environment:** They collect data about their surroundings such as sensor inputs or observations.
- **Defining the problem:** They clearly understand the problem including the starting point, the available actions and the desired goal.
- **Exploring different possibilities:** They consider various ways to solve the problem and evaluate which approach is likely to succeed.
- **Evaluating and deciding:** Once they explore options, they assess the outcomes and pick the best course of action based on factors like time, resources and success likelihood.
- **Learning and adapting:** Many problem-solving agents can learn from past experiences, improving their decision-making abilities over time.

Goal Formulation

- The first step is to define the **goal**. A goal is a desired state that the agent wishes to reach. Goals must be:
 - **Well-defined**
 - **Achievable**
 - **Measurable**
- For example, for a delivery robot, the goal might be “Deliver the package to Building B.”

Problem Formulation

- Once a goal is set, the agent formulates the problem using the following components:
- **Initial State:** Where the agent starts.
- **Actions (Successor Function):** What can be done from a given state.
- **Transition Model:** Describes the result of performing an action.
- **Goal Test:** Determines whether the goal has been reached.
- **Path Cost:** Measures the cost of a path (e.g., distance, time).

Search for Solutions

- The agent uses search algorithms to find a path from the initial state to a goal state. This step involves:
- Building a **search tree**
- Exploring nodes (states)
- Evaluating paths using cost or heuristic

Execution

- Finally, the agent executes the selected sequence of actions to achieve the goal.
- It may replan if the environment changes unexpectedly.

Types of Problems in AI

AI systems can face different types of problems. These problems are categorized based on their impact on the system's performance and how they can be handled:

1. Ignorable Problems

- These are minor issues that have little or no effect on the AI system's overall performance. They are often harmless and do not require immediate attention.

Examples:

- Small inaccuracies in predictions that don't affect the final result like a slight error in image classification.
- Minor issues in data preprocessing that don't change the outcome.
- Handling: Ignored as they don't interfere with the system's primary function.

2. Recoverable Problems

- Recoverable problems are those where the AI system encounters an issue but can be fixed with intervention either automatically or manually, such as error-handling functions.

Examples:

- Missing data that can be filled in using statistical methods.
- System crashes that can be fixed by restoring from a backup.
- Handling: Requires action like automated error handling, **retraining** the model or system recovery through checkpoints.

Example

- What makes the following code ?

```
for(int n=10; n>=10;n++){  
    System.out.print("need actions");  
}
```

3. Irrecoverable Problems

- These are severe issues that cause permanent damage or failure, making it impossible for the system to recover. They can lead to significant performance loss.

Examples:

- Corrupted training data that causes bias and reduces the model's effectiveness.
- Adversarial attacks that make the model untrustworthy.
- Overfitting, where the model becomes too specialized and cannot adapt to new data.
- Handling: Requires a complete rebuild of the system including retraining the model or updating the data to eliminate underlying issues.

Steps in Problem Solving in AI

- AI problem solving follows a structured, logical process similar to human thinking, its steps include:
- **Problem Definition:** First, the problem needs to be clearly defined. This includes understanding the inputs, constraints and what the solution should look like.
- **Problem Analysis:** Once the problem is defined, it's analyzed in more detail. This helps in understanding its limitations and possible solutions.
- **Knowledge Representation:** All the important information is organized so the AI can understand and work with it. This could include creating graphs or using databases.
- **Problem Solving:** AI uses appropriate methods to solve the problem. This often means comparing different strategies to find the most efficient one.
- **Testing and Evaluation:** Finally, after the solution is implemented, testing and evaluation ensure the solution meets all requirements and performs as expected.

Components of Problem Formulation in AI

Effective problem-solving in AI is dependent on several important components:

- **Initial State:** This is the starting point of the problem where the AI begins its process. It sets the context and helps identify how the agent will approach the challenge.
- **Action:** At this stage, AI identifies all the possible actions it can take from the initial state. Each action has an impact on how the system moves closer to solving the problem.
- **Transition:** This refers to how the system changes from one state to another after an action is taken. Transition modeling helps show how the agent's actions influence the next steps.
- **Goal Test:** Once an action is taken, AI checks if it has reached its goal. If the goal is achieved, the problem-solving process stops and the solution is considered complete.
- **Cost Function:** This step assigns a numerical value to the cost of achieving the goal. The cost can include resources like time, energy or money and helps decide the most efficient way to reach the goal.

Take delivery robot

- Could you list Components
 - Like the state?
 - Action ?
 - Transition ?
 - Goal test and cost ?

Techniques for Problem Solving in AI

- Several techniques are available to solve different types of problems in AI. The choice of technique depends on the nature of the problem and the resources available.

1. Search Algorithms

- Search Algorithms are important for exploring possible solutions. There are two main categories:
- **Uninformed Search:** These algorithms such as breadth-first and depth-first search, don't use additional knowledge about the problem and explore all possible paths.
- **Informed Search:** Algorithms like A* use heuristics or domain-specific knowledge to guide the search toward more promising paths making the search process more efficient.

2. Constraint Satisfaction Problems (CSP)

- In CSPs the goal is to find solutions that satisfy a set of constraints.
- Techniques like backtracking, constraint propagation and local search are used to explore the solution space efficiently while ensuring that all constraints are met.

3. Optimization Techniques

- AI often addresses optimization problems where the goal is to find the best possible solution among many options. Techniques include:
- Linear programming for solving problems involving linear relationships.
- Dynamic programming for breaking down complex problems into smaller subproblems.
- Evolutionary algorithms like genetic algorithms that simulate natural selection to find optimal solutions.

4. Machine Learning

- Machine learning allows AI systems to learn from data and improve over time. The three major learning types are:
- Supervised Learning: Learning from labeled data to make predictions or classifications.
- Unsupervised Learning: Discovering patterns in data without labeled outputs.
- Reinforcement Learning: Learning through trial and error, receiving feedback in the form of rewards or penalties.

5. Natural Language Processing (NLP)

- Natural Language Processing (NLP) helps AI systems understand and generate human language.
- Techniques like tokenization, Part-of-speech (PoS) and named entity recognition are important for chatbots, language translation and text summarization.

Advantages of AI in Problem Solving

- **Efficiency:** It can solve problems much faster than humans, processes large amounts of data and identifying patterns quickly.
- **Accuracy:** With the right data and models, AI can make decisions with high accuracy, especially in tasks like medical diagnoses or image recognition.
- **Automation:** It automates repetitive tasks, freeing up human time for more complex and creative work.
- **Adaptability:** AI systems, particularly those using machine learning can improve over time, adapting to new data and evolving challenges.
- **Cost-Effectiveness:** Once implemented, it can reduce costs by simplifying operations, improving productivity and minimizing human error.

Challenges in AI Problem Solving

- Despite its many advantages, it faces several challenges:
- Complexity: Some problems are highly complex, requiring significant computational resources and time to solve.
- **Data Quality:** AI systems depend on data quality and poor or biased data can lead to inaccurate or misleading outcomes.
- Interpretability: Many AI models, especially deep learning, lack transparency, making it hard to understand how decisions are made.
- Ethics and Bias: AI can conserve biases in data which leads to unfair or unethical outcomes, ensuring fairness is a key challenge.

Problem Solving Agents summary

- In the world of Artificial Intelligence, a **Problem Solving Agent** is a type of goal-based agent that uses **atomic representations** meaning it views the world as a set of states that are indivisible "black boxes."
- These agents don't just react; they plan ahead to find a sequence of actions that leads to a desired goal.

The 4-Step Process

- To solve a problem, an agent typically follows this linear cycle:
 - 1. Goal Formulation:** Defining what the "success" state looks like. (e.g., "I want to be in Bucharest.")
 - 2. Problem Formulation:** Deciding which actions and states to consider given the goal. This involves "abstracting" away unnecessary details (like the color of the road or the brand of the car).
 - 3. Search:** The process of looking for the best sequence of actions to reach the goal. The agent simulates different paths in its "head" before acting.
 - 4. Execution:** Once a solution is found, the agent performs the actions one by one.

Defining the "Problem"...

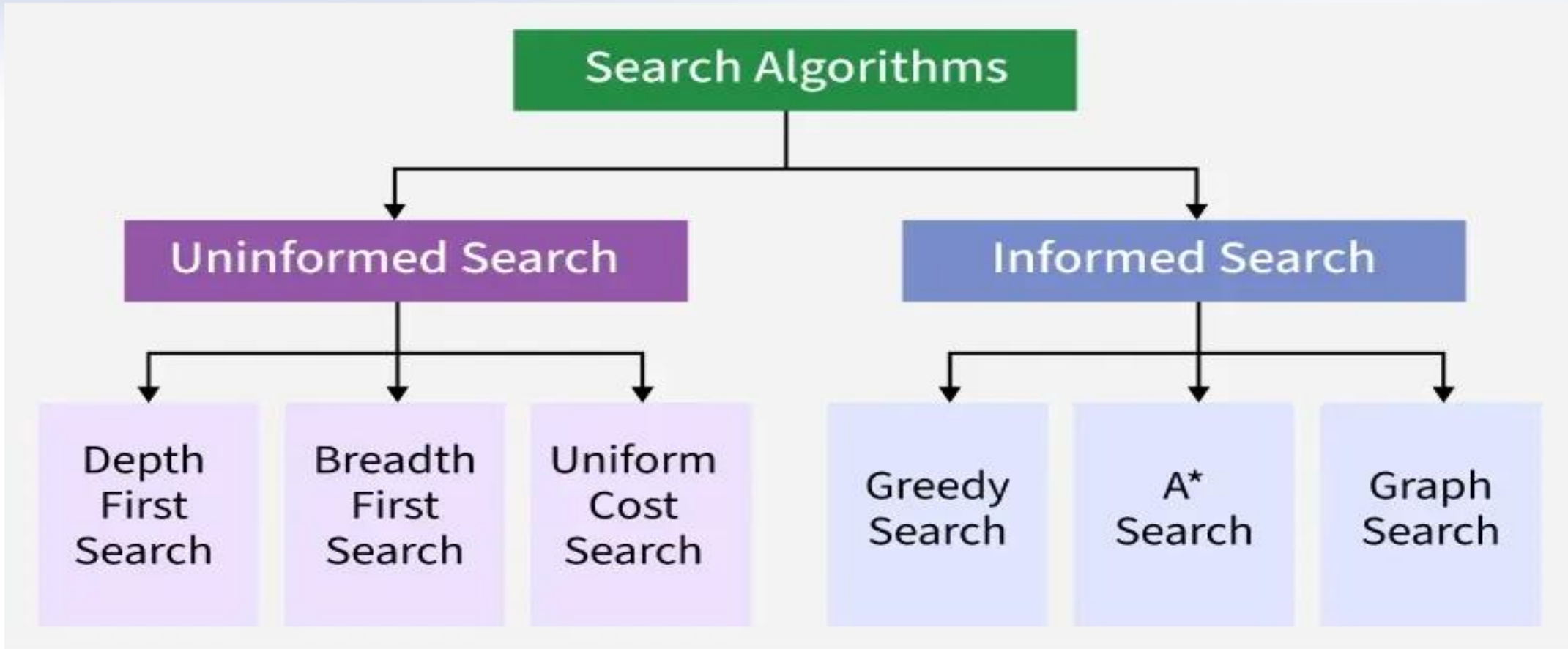
- Technically, a problem is defined by five specific components:

Component	Description	Example (Navigation)
Initial State	Where the agent starts.	Being in Addis.
Actions	The possible moves available.	{Go to Mojo, Go to Adama}
Transition Model	What happens after an action?	If I take "Go to Mojo," I am now in Mojo.
Goal Test	Determines if the current state is the goal.	Is <code>current_location == Adama</code> ?
Path Cost	A numeric cost (distance, time, fuel).	The mileage between cities.

Search Algorithms in AI

- Search algorithms in AI help find solutions by exploring possible paths or options in a problem space.
- AI uses them in tasks like
 - pathfinding,
 - decision making and
 - game playing.
- These algorithms work by searching through a set of possibilities to reach a goal, either blindly without extra information or with guidance using heuristics.

Types of search algorithms



Cont....

1) Uninformed Search Algorithms

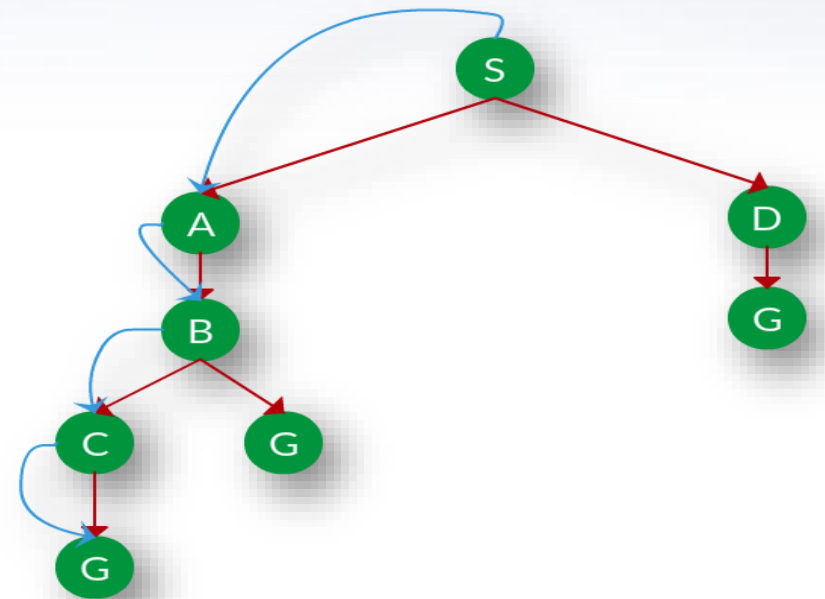
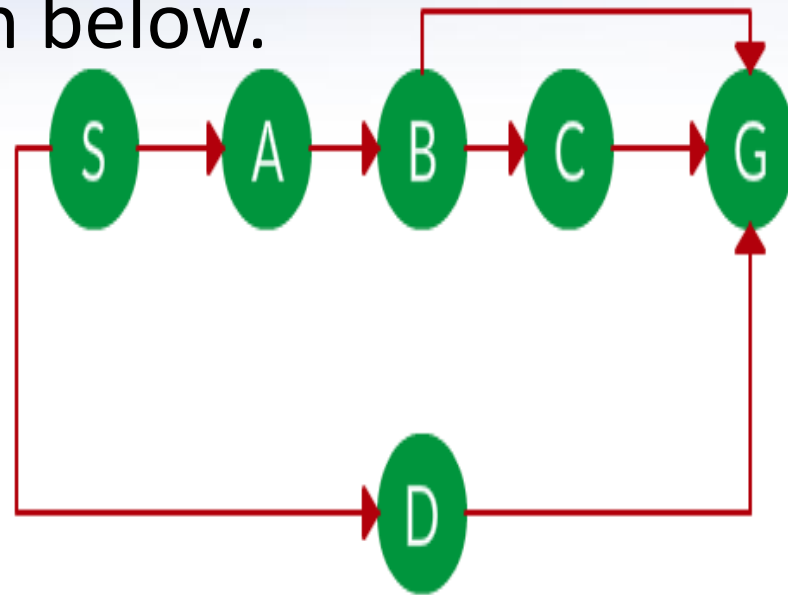
- Uninformed search also called blind search explores the search space without any domain specific knowledge or **heuristics**.
- It treats all **nodes** equally and chooses which path to explore next based solely on general rules like
 - node depth or
 - path cost.

1. Depth First Search

- **Depth-First Search (DFS)** is a fundamental uninformed search algorithm in Artificial Intelligence (AI) used for traversing or searching tree and graph data structures.
- Depth First Search explores paths by going as deep as possible along one direction before backtracking.
- It uses a stack or recursion to keep track of the path.
- DFS is memory efficient compared to BFS since it doesn't need to store all **siblings** at each level.
- However it is not guaranteed to find the shortest path and may get stuck in an infinite loop if the search tree is deep or contains cycles unless depth limits or visited checks are applied.

For Example:

- Which solution would DFS find to move from node S to node G if run on the graph below.



How DFS Works in AI

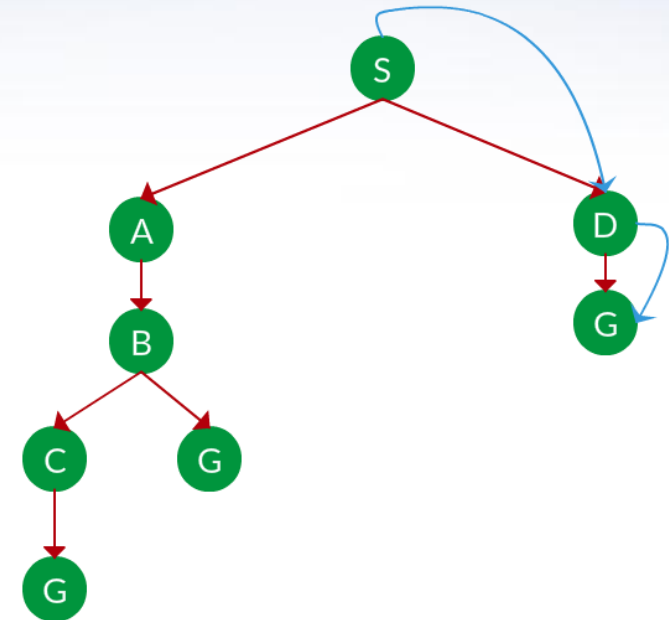
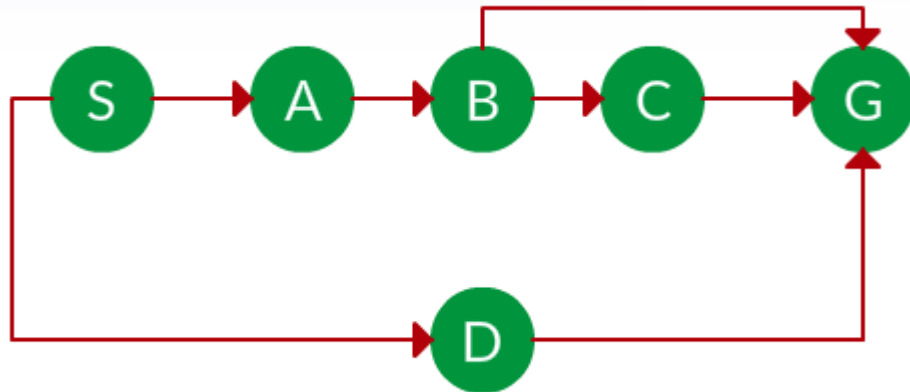
- **DFS** is like venturing into a maze and following one path to its end before trying another. It uses a stack (either explicit or implicitly via recursion) to manage the nodes that need to be visited.
- **Initialization**: Start at an initial node (often the root in a tree, or an arbitrary node in a graph) and mark it as visited.
- **Exploration**: Push the starting node onto a stack. While the stack is not empty, pop the top node.
- **Deep Dive**: For the current node, explore one of its unvisited neighbors. Mark that neighbor as visited and push it onto the stack.
- **Backtracking**: Continue this process, moving deeper into the graph. **If a node** has no unvisited neighbors (a "dead end"), the algorithm backtracks by popping nodes from the stack until it finds a node with an unexplored branch.
- **Termination**: The process continues until the goal is found, or the stack is empty, meaning all reachable nodes have been visited.

2. Breadth First Search

- Breadth First Search is a fundamental search algorithm that explores all possible paths level by level.
- It begins from the root node and explores all neighboring nodes before moving to the next level of nodes.
- BFS is complete and guarantees finding the shortest path if each move has the same cost.
- However its main **drawback** is high memory usage as it stores all nodes at the current level before moving deeper which can grow rapidly for large or complex problems.

Example:

- Which solution would BFS find to move from node S to node G if run on the graph below.



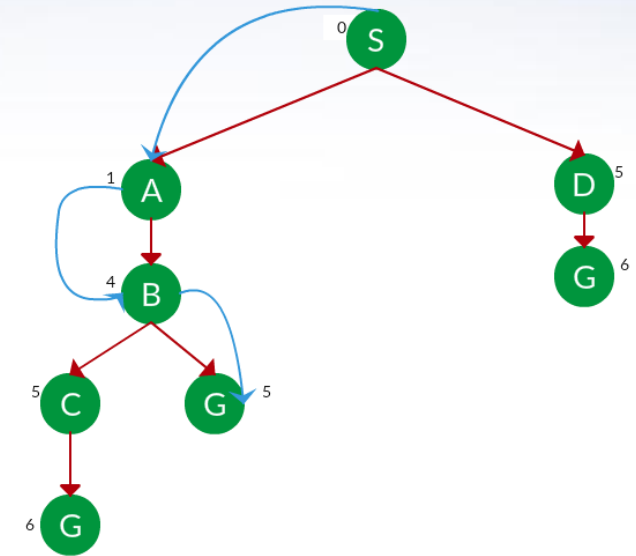
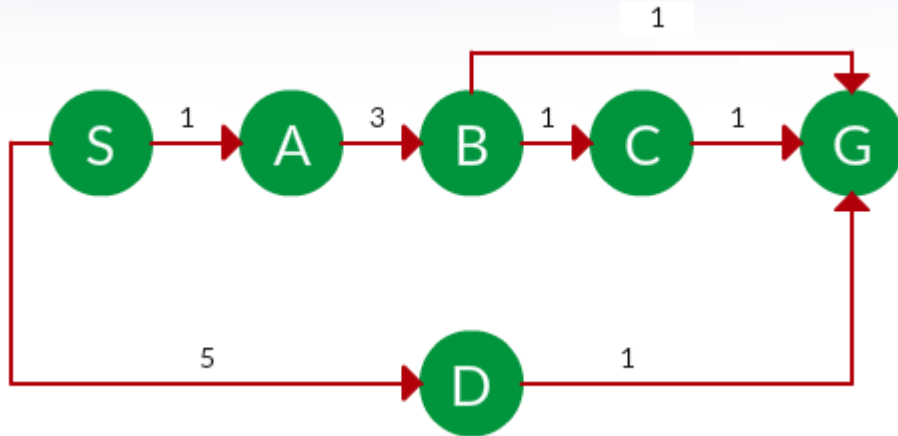
As BFS traverses the tree shallowest node first it would always pick the shallower branch until it reaches the solution or it runs out of nodes and goes to the next branch. Path: S->D->G

3. Uniform Cost Search

- Uniform Cost Search is similar to BFS but takes the cost of each move into account. It always expands the node with the lowest cumulative path cost from the start.
- This makes **UCS** optimal and complete and useful when actions have different costs such as in navigation systems.
- It uses a **priority queue** to manage the frontier and ensures the cheapest path is always chosen next.

For Example:

- Which solution would UCS find to move from node S to node G if run on the graph below.



The cost of each node is the cumulative cost of reaching that node from the root and based on the UCS strategy the path with the least cumulative cost is chosen.

Path: S->A->B->G

Informed Search Algorithms

- **Informed search algorithms** in AI are search methods that use extra knowledge, called heuristics, to prioritize which paths to explore.
- By estimating how close each possible step is to the goal, these algorithms can find solutions more quickly and efficiently than uninformed or “blind,” search.
- They are **widely** used in AI for tasks like pathfinding and puzzle solving because they help navigate large, complex search spaces.

1 Greedy Search

The greedy search algorithm (more formally, Greedy Best-First Search in AI) is an informed search algorithm that expands the node which appears closest to the goal based on a heuristic function (an educated guess or estimate of the remaining cost).

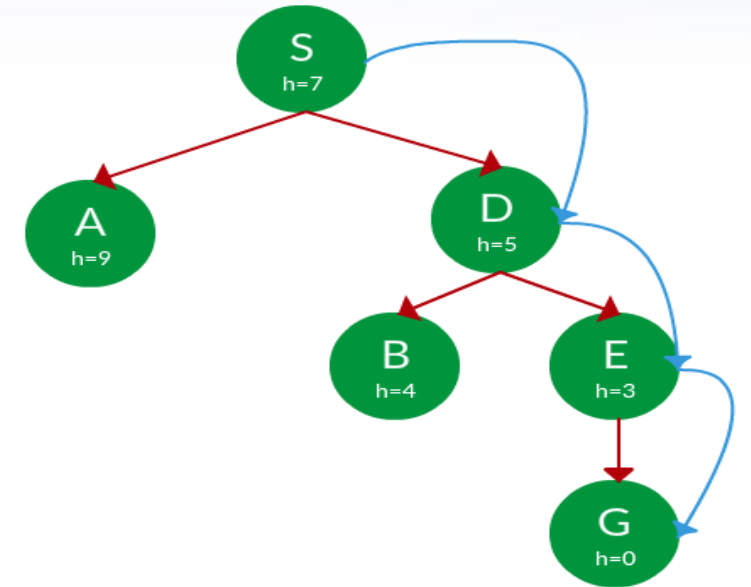
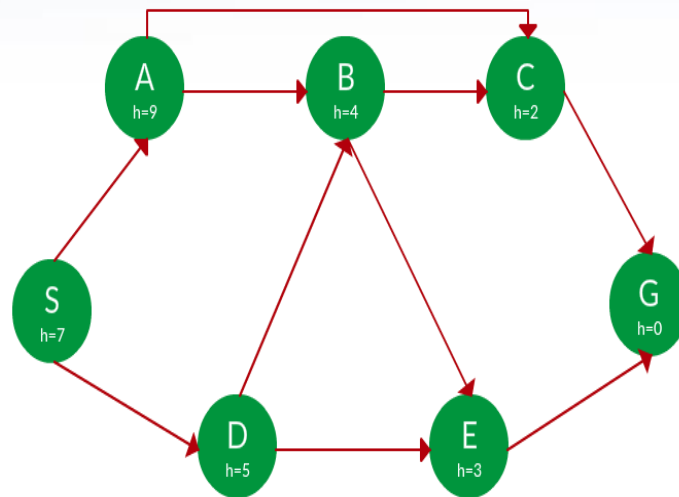
It is called "greedy" because it focuses solely on immediate, local gains without considering the long-term consequences or the total path cost already incurred..

How It Works

- In a search tree or graph, a Greedy algorithm uses a heuristic function, often denoted as $h(n)$.
- Where:
 - ✓ n = current node
 - ✓ $h(n)$ = estimated cost from node n to the goal
 - ✓ It does **NOT** consider the cost from start to current node.
It only looks at how close it *seems* to the goal.
- This function estimates the cost from the current state **n** to the goal.
 - 1) **Evaluate**: Look at all possible next steps from the current node.
 - 2) **Select**: Choose the node that appears "closest" to the goal according to the heuristic.
 - 3) **Repeat**: Move to that node and repeat the process until the goal is reached or no further moves are possible.

For Example:

- Find the path from S to G using greedy search, the heuristic values h of each node below the name of the node.



Starting from S we can traverse to A($h=9$) or D($h=5$). We choose D as it has the lower heuristic cost.

Now from D we can move to B($h=4$) or E($h=3$).

We choose E with a lower heuristic cost.

Finally from E we go to G($h=0$). This entire traversal is shown in the search tree above, in blue.

Path: S- $\rightarrow$ D- $\rightarrow$ E- $\rightarrow$ G

2. A* Tree Search:

- The A* (A-star) algorithm is a widely used informed search algorithm in AI that efficiently finds the shortest path between two points.
- It works by using a heuristic function to prioritize the most promising paths, making it more efficient than uninformed search methods..
- This can result in duplicate work and a larger search space, especially in graphs with cycles.
- Although it's simpler to implement A* tree search may be less efficient and is typically used when the search structure is naturally a tree or when memory constraints prevent maintaining a closed list.

How A* Tree Search Works

- The A* algorithm uses a cost function for each node to determine the order in which to explore the search space.

- The function is defined as: $f(n)=g(n)+h(n)$

g(n): The actual cost (e.g., distance, time) incurred to reach the current node from the start node.

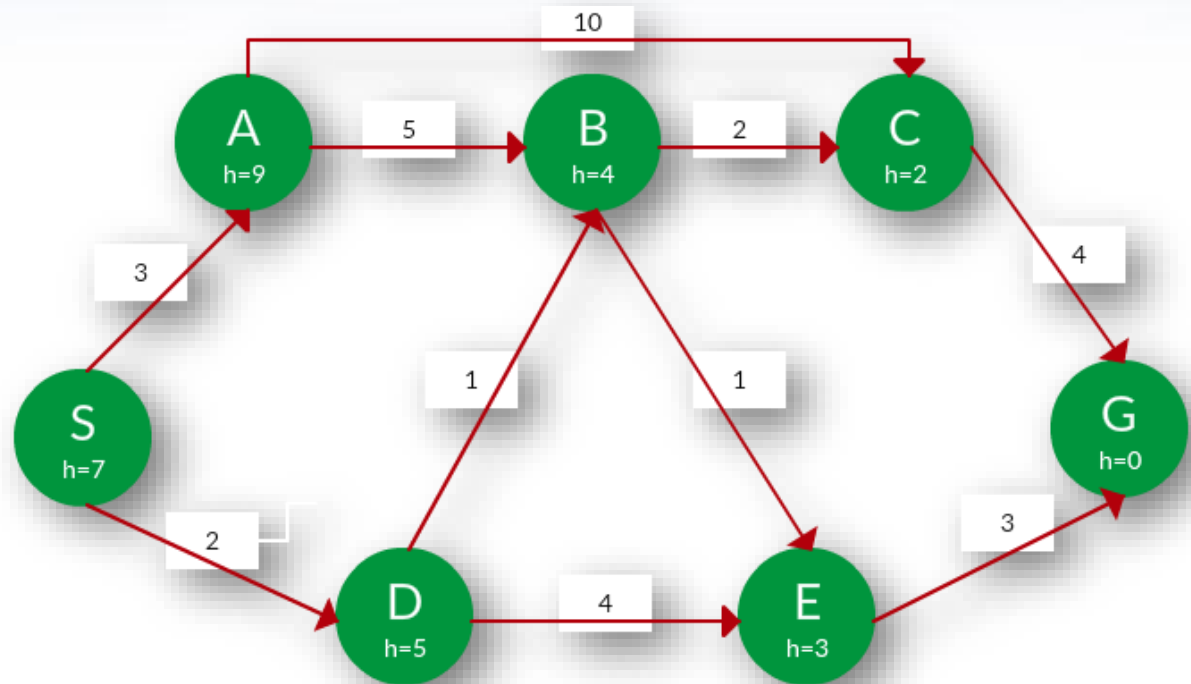
H(n): The heuristic estimate of the cost from node to the goal node. This is an intelligent guess that guides the search towards the goal.

F(n): The total estimated cost of the path from the start node to the goal node through node.

For Example:

- Find the path to reach from S to G using A* search. .

Starting from S the algorithm computes $g(x) + h(x)$ for all nodes in the fringe at each step choosing the node with the lowest sum. The entire work is shown in the table below. It search like:



Cont....

Path	$h(x)$	$g(x)$	$f(x)$
S	7	0	7
S -> A	9	3	12
S -> D ✓	5	2	7
S -> D -> B ✓	4	$2 + 1 = 3$	7

S -> D -> E	3	$2 + 4 = 6$	9
S -> D -> B -> C ✓	2	$3 + 2 = 5$	7
S -> D -> B -> E ✓	3	$3 + 1 = 4$	7
S -> D -> B -> C -> G	0	$5 + 4 = 9$	9
S -> D -> B -> E -> G ✓	0	$4 + 3 = 7$	7

- Path: S->D->B-> G-> E and Cost: 7

Avoiding Repeated States

- Avoiding repeated states in AI search algorithms, such as BFS, DFS, or A*, is crucial for preventing cycles and reducing computational complexity.
- It is achieved by implementing a **closed list** (or hash table) to store visited states, ignoring already visited states, and avoiding moves that immediately reverse the previous action.

Methods for Avoiding Repeated States

- **Closed List / Explored Set:** Maintain a data structure (e.g., hash set) to keep track of all states that have already been expanded. Before adding a new state to the "fringe" (or "open list"), check if it exists in the closed list.
- **Cycle Detection:** When generating child nodes, check if any child node already exists on the current path from the root to the current node to avoid infinite loops.
- **Parent-Child Restriction:** Do not allow the algorithm to return directly to the parent state (e.g., if you moved "Up" to reach the current state, do not allow a "Down" move immediately).
- **Graph Search:** Utilize graph search instead of tree search, as graph search inherently tracks visited states, preventing the exponential explosion caused by re-expanding the same nodes.

Implementation Strategies

- Data Structures: Use hash tables or sets for fast lookups ($O(1)$ average time complexity) rather than lists or arrays to store the closed set, as the choice of data structure significantly impacts runtime.
- Memory vs. Time Trade-off: While storing all visited states requires more memory, it prevents redundant computations, allowing algorithms to solve complex problems efficiently.
- Heuristics: Informed search algorithms like A* use heuristic functions ($f(n)=g(n)+h(n)$) to prioritize paths, naturally guiding the search away from cycles and towards the goal.

Common Pitfalls

- Revisiting states: Without a closed list, the same state can be reached through different paths, leading to exponential, redundant work.
- Infinite loops: In search spaces with cycles, algorithms like DFS can get trapped without cycle detection.

Read the following

1. Constraint Satisfaction Search
2. Games as Search Problems

Project assignment

- Find problems around you that can be solved by AI and prepare a title which fits with intelligent agent or machine learning algorithm and submit the title in one week from this day (26-06-2018 EC)
 - After submitted the title and accepted you should implement the solution using your own approach or
 - 1) Using intelligent agent concept or
 - 2) using any of the machine learning algorithm
- Total project value (20%)

~~~~~

**This is all about chapter 3**